Final Report Contextual Word Sense Disambiguation Using the T5 Transformer Model

Course: 1164 EEE G513 - Machine Learning for Electrical Engineering

Jaya Krishna B (2022A8PS1439G)
Dinesh Kumar P (2022AAPS0659G)

1 Introduction

T5 (Text-To-Text Transfer Transformer) is a transformer model. It is based on the encoder-decoder architecture of transformers and is designed to handle a wide range of natural language processing tasks. T5 converts every NLP problem into a text-to-text format, which means both the input and output are treated as text strings, making it highly versatile for tasks like translation, summarization, and classification. Pre-trained on a multi-task mixture of supervised and unsupervised tasks. By appending a distinct prefix to the input that corresponds to each task, T5 performs well on a range of unconventional tasks. We shall be fine tuning the T5 model to perform word sense disambiguation using the WiC dataset, which is a part of the SuperGLUE benchmark.

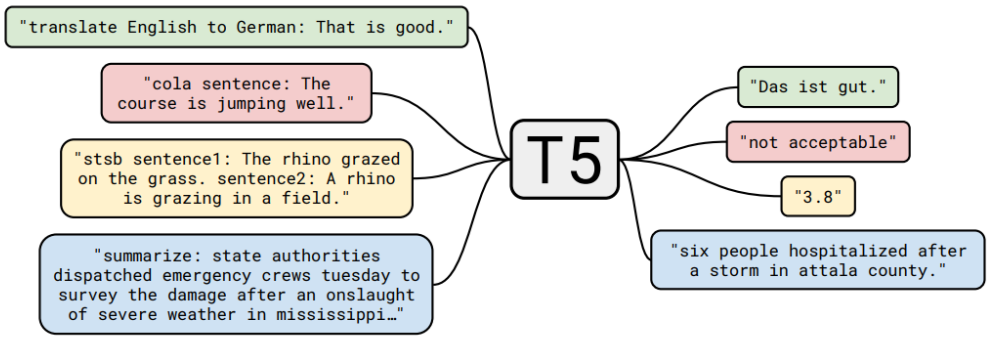

Figure 1: Different tasks carried out by T5 model

2 Preprocessing

The preprocessing of the dataset involved several systematic steps to prepare the WiC dataset for fine-tuning the T5 model. Initially, the WiC dataset was loaded from the SuperGLUE benchmark using the Hugging Face ‘datasets‘ library. Each data sample in the dataset consists of a target word and two sentences where the word is used. The goal is to determine whether the target word holds the same meaning in both sentences, with labels indicating ”1” for ”Yes” and ”0” for ”No.”

To facilitate this task, a preprocessing function was implemented to reformat the input data. For each example, the function extracted the target word, the two sentences, and the label. It then constructed input strings in the format: ”Sentence 1: sentence1 Sentence 2: sentence2 Word: word” These strings were paired with corresponding target texts: ”Yes” for label 1 and ”No” for label 0. This approach ensures that the inputs are suitable for the T5 model’s text-to-text framework.

Following preprocessing, the dataset was tokenized using the T5 tokenizer. The tokenization process involved truncating or padding the input and target texts to a fixed length to make them compatible

Dataset	Instances	Nouns (%)	Verbs (%)	Unique Words
Training	5,428	49%	51%	1,256
Dev	638	62%	38%	599
Test	1,400	59%	41%	1,184

Table 1: Summary of Instances, Nouns, Verbs, and Unique Words in Datasets.

with the model's input requirements. The tokenized dataset was then converted to PyTorch tensors and formatted for use with 'DataLoader' objects to enable efficient batching during training, validation, and testing.

This preprocessing ensured that the dataset was properly structured, with well-defined input-output pairs, making it ready for fine-tuning the T5 model with additional LoRA adapters for parameter-efficient training.

3 Transformers

The transformer model revolutionized deep learning, particularly in natural language processing, by enabling parallel sequence processing instead of the sequential approach used by RNNs, GRUs, and LSTMs. These earlier models struggled with long-term dependencies due to issues like vanishing gradients and a limited "attention window." Transformers addressed this by introducing self-attention, which allows each token to dynamically focus on all other tokens in a sequence, enabling better context understanding. Although transformers don't have an infinite attention window, they excel at modeling long-range relationships within the input sequence.

3.1 Input embedding

Feeding input into a word embedding layer is the initial stage. One way to conceptualize a word embedding layer is as a lookup table that retrieves a learned vector representation for every word. Since neural networks learn by using numbers, every word may be represented by a vector with continuous values.

3.2 Positional Encoding

Injecting positional information into the embeddings is the next stage. We need to include some positional information in the input embeddings because the transformer encoder lacks recurrence, unlike recurrent neural networks. Positional encoding is used for this.

3.3 Encoder layer

All input sequences must be mapped into an abstract continuous representation by the encoder layers in order to store the learned information for the full sequence. Two sub-modules are included: multi-headed attention and a completely integrated network. Each of the two sublayers has residual connections surrounding it, which are followed by layer normalization.

3.4 Decoder layer

The task of the decoder is to produce text sequences. The sub-layers of the encoder and decoder are comparable. It features a pointwise feed-forward layer, two multi-headed attention layers, residual connections, and layer normalization following each sub-layer. Each multi-headed attention layer performs a distinct function, yet these sub-layers operate similarly to the encoder's layers. A linear layer that serves as a classifier and a softmax to obtain the word probabilities complete the decoder.

3.5 Self-Attention

Self-attention is a mechanism that enables a model to focus on different parts of an input sequence when processing it, capturing relationships and dependencies between tokens, regardless of their distance in

the sequence. Each token in the input is transformed into three vectors: query, key, and value, using learned weight matrices. The model computes attention scores by comparing the query vector of each token with the key vectors of all tokens in the sequence. These scores, which represent the relevance of one token to another, are normalized using the softmax function. The normalized scores are then used to compute a weighted sum of the value vectors, producing a refined representation for each token. By dynamically adjusting focus based on context, self-attention allows the model to understand both local and global dependencies, making it highly effective for tasks like language modeling, translation, and text summarization.

4 Model architecture

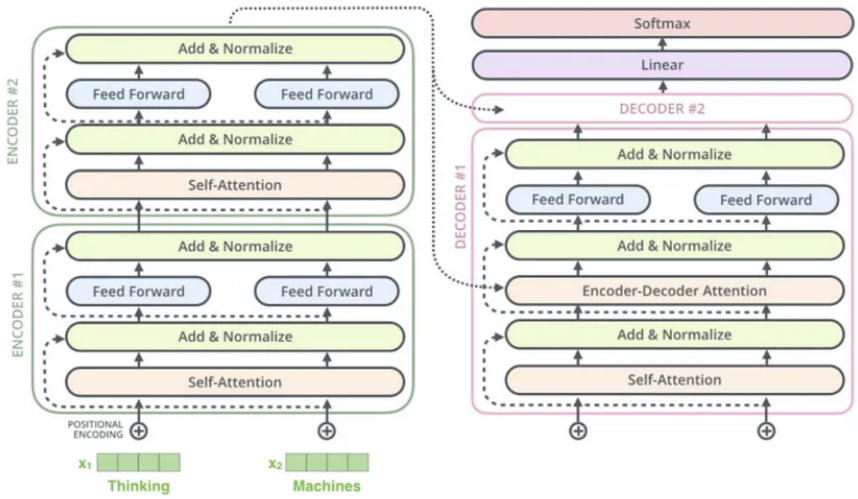

Figure 2: T5 Model architecture

The diagram represents the architecture of a transformer-based model, which underlies the T5 (Text-to-Text Transfer Transformer) framework. It showcases the encoder-decoder structure, where the input sequence (e.g., "thinking machines") is first passed through a stack of encoder layers. Each encoder layer consists of self-attention mechanisms and feed-forward networks, along with Add and Normalize layers, which stabilize training. Positional encodings are added to the input embeddings to incorporate sequence order.

The encoded representation is then processed by the decoder stack, which similarly contains self-attention, feed-forward layers, and encoder-decoder attention. The decoder's self-attention attends to the previously generated tokens, while the encoder-decoder attention links the encoded input with the current output generation. At the top of the decoder, the final output logits are generated using a linear layer followed by a softmax function to predict the next token in the sequence. This iterative process continues until the entire output sequence is generated.

This architecture enables T5 to perform various natural language processing tasks in a unified "text-to-text" format, where inputs and outputs for tasks like translation, summarization, and classification are handled as text strings.

5 Optimal hyperparameter setting

Using Optuna, a state-of-the-art hyperparameter optimization framework, we systematically explored a wide range of hyperparameters to fine-tune the model. The goal was to maximize validation accuracy by identifying the best combination of hyperparameters for our task. The optimization process utilized a tree-structured Parzen estimator (TPE) as the underlying sampling algorithm to efficiently navigate the search space. After multiple trials, the optimal configuration was determined as follows:

Optimal Parameters

- **Learning Rate (lr)**: 9.76×10^{-5}
- **Batch Size**: 16
- **Dropout Rate (dropout)**: 0.45
- **Optimizer**: Adam

These hyperparameters were selected based on their ability to maximize validation accuracy during training. The learning rate ensures gradual updates to the model weights, the batch size balances computational efficiency and generalization, and the dropout rate effectively mitigates overfitting.

6 Results

The model was trained for 20 epochs with the given hyperparameters. The accuracy and loss curves for both training and validation data are shown in Figures 3 and 4, respectively.

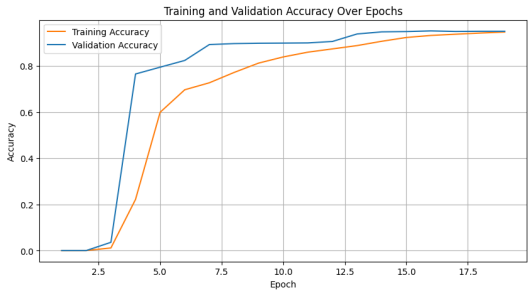

Figure 3: Accuracy curve of Train/Validation data

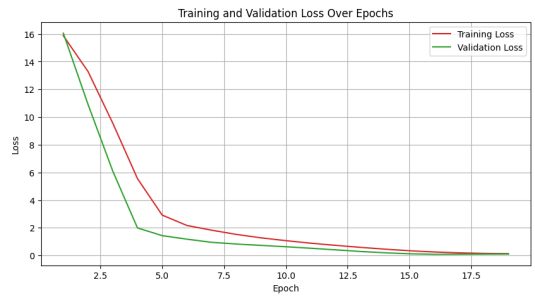

Figure 4: Loss curve of Train/Validation data

The performance comparison between the model with and without additional layers is summarized in Table 2, showcasing the accuracy percentages for training, validation, and test datasets.

	Train (%)	Validation (%)	Test (%)
Without Layer	95.81	95.25	64.21
With Layer	81.13	80.02	65.71

Table 2: Performance comparison between the model with and without additional layers.

7 Error Analysis

Enhancing the dataset by paraphrasing sentences is one possible method to increase accuracy. The model can be exposed to a greater range of language structures and idioms that convey comparable ideas by producing several paraphrases for every sentence. This lessens the model's confusion caused by lexical variability and semantic ambiguity, which are particularly problematic when a word has several meanings. By allowing the model to better accommodate sentence changes that might not have been present in the original training data, paraphrasing can help improve the model's generalization capabilities and possibly improve performance in ambiguous or edge-case situations.